Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
CE1113 Sistemas Empotrados
Prof. Dr.-Ing. Jeferson González Gómez

Taller 3: Herramientas de Construcción Automatizada

Make, Autotools y CMake

1. Introducción

En el desarrollo de software embebido es fundamental dominar las herramientas que automatizan la construcción de programas y bibliotecas. Este taller abarca tres herramientas ampliamente utilizadas: **GNU Make**, **GNU Autotools** y **CMake**. Cada parte del taller se desarrolla sobre el **mismo código fuente de ejemplo**: una biblioteca mínima *libhello* con una sola función, cuya simplicidad permite concentrarse en las herramientas sin distracciones.
Al finalizar el tutorial, el estudiante aplicará los conocimientos adquiridos de forma independiente, construyendo una biblioteca matemática completa con las tres herramientas.

Código fuente de referencia (tutorial)

La estructura de archivos utilizada en los tres tutoriales es:

```
libhello/
  include/
    say.h
  lib/
    say.c
  src/
    main.c
```

include/say.h

```c
#ifndef SAY_H
#define SAY_H

void say_hello(void);

#endif /* SAY_H */
```

lib/say.c

```c
#include <say.h>
#include <stdio.h>

void say_hello(void)
{
    printf("Hello, World!\n");
}
```

src/main.c

```c
#include <say.h>

int main(void)
{
    say_hello();
    return 0;
}
```

2. Parte I: GNU Make

2.1. ¿Qué es Make?

GNU Make es una herramienta de construcción que determina automáticamente qué partes de un proyecto necesitan recompilarse y ejecuta los comandos necesarios para hacerlo. Su principal ventaja respecto a compilar manualmente con `gcc` es que **solo recompila lo que cambió**, evitando reconstruir el proyecto completo ante una modificación pequeña. Esto es especialmente valioso en proyectos con muchos archivos fuente.
Make lee un archivo llamado **Makefile** en el que el desarrollador describe las relaciones entre archivos y los comandos para producirlos. A diferencia de Autotools o CMake, Make no genera sus propios archivos de construcción: el desarrollador los escribe directamente.

2.2. Estructura de un Makefile

Un Makefile se compone de tres elementos fundamentales: **variables**, **reglas** y **directivas**.

Variables (macros)

Las variables almacenan valores reutilizables. Se definen con `=` o `:=` y se referencian con `$(NOMBRE)`:

```
CC      = gcc        # compilador de C
AR      = ar         # archivador (para bibliotecas .a)
CFLAGS  = -Wall -O2 # banderas del compilador
LDFLAGS = -lm        # banderas del enlazador
```

Las variables de *toolchain* (`CC`, `AR`, `CXX`, `STRIP`, etc.) tienen especial importancia en sistemas embebidos: al cambiar el *cross-compiler* (por ejemplo a `arm-linux-gnueabihf-gcc`) basta con modificar `CC` y todos los comandos del Makefile se actualizan automáticamente, sin tocar las reglas.

Reglas

Una regla describe cómo producir un **objetivo** (*target*) a partir de sus **dependencias** y qué **comandos** ejecutar. La indentación de los comandos **debe** ser una tabulación (`<TAB>`), nunca espacios.

```
objetivo: dependencia1 dependencia2 ...
<TAB>comando1
<TAB>comando2
```

Make evalúa si el objetivo es más reciente que sus dependencias. Si alguna dependencia es más nueva (o el objetivo no existe), ejecuta los comandos; de lo contrario, no hace nada. Este mecanismo de **timestamps** es lo que evita recompilaciones innecesarias.

Variables automáticas

Dentro de los comandos, Make ofrece variables automáticas que evitan repetir nombres:

Variable	Significado
`$@`	Nombre del objetivo de la regla actual.
`$<`	Primera dependencia de la regla.
`$^`	Lista de todas las dependencias (sin duplicados).
`$?`	Dependencias más recientes que el objetivo.

Reglas de patrón

Las **reglas de patrón** generalizan la compilación de archivos que siguen el mismo esquema:

```
# Cualquier .o se produce a partir del .c del mismo nombre
%.o: %.c
$(CC) $(CFLAGS) -c $< -o $@
```

Objetivos PHONY

Un objetivo **PHONY** no corresponde a ningún archivo; siempre se ejecuta cuando se invoca. Se declaran con `.PHONY` para que Make no los confunda con archivos reales:

```
.PHONY: all clean

clean:
rm -f *.o libhello.a libhello.so
```

Makefiles recursivos

En proyectos con múltiples directorios es práctica común usar un **Makefile principal** que invoca Makefiles secundarios mediante `$(MAKE) -C <directorio>`. Usar `$(MAKE)` en lugar de `make` garantiza que se propagen correctamente las opciones del invocador (como `-j` para compilación en paralelo).

2.3. Make: modo usuario

Cuando se descarga un proyecto que ya incluye un `Makefile`, el flujo es simple:

1. Descomprimir y entrar al directorio del proyecto.
2. `make` — construye los objetivos por defecto.
3. `make install` — instala (si el Makefile lo define).
4. `make clean` — elimina los archivos generados.

A diferencia de Autotools y CMake, no existe un paso de configuración previo.

2.4. Aplicación: *libhello* con Makefiles

A continuación se aplican los conceptos anteriores para construir *libhello* usando tres Makefiles coordinados: uno en `lib/` que genera las versiones estática (`.a`) y dinámica (`.so`), uno en `src/` que compila la aplicación en ambas variantes, y un Makefile principal que los coordina.

```
libhello/
  Makefile              <- Makefile principal (recursivo)
  include/
    say.h
  lib/
    Makefile            <- genera libhello.a y libhello.so
    say.c
  src/
    Makefile            <- genera saludo_e y saludo_d
    main.c
```

```
bin/                    <- saludo_e, saludo_d (generados)
```

Makefile principal (libhello/Makefile):

```
CC  = gcc
AR  = ar
.PHONY: all lib src clean

all: lib src

lib:
$(MAKE) -C lib

src: lib
$(MAKE) -C src

clean:
$(MAKE) -C lib clean
$(MAKE) -C src clean
```

lib/Makefile:

```
CC     = gcc
AR     = ar
CFLAGS = -Wall -fPIC -I../include
LIB    = libhello

.PHONY: all clean

all: $(LIB).a $(LIB).so

say.o: say.c
$(CC) $(CFLAGS) -c $< -o $@

$(LIB).a: say.o
$(AR) rcs $@ $^

$(LIB).so: say.o
$(CC) -shared -o $@ $^

clean:
rm -f say.o $(LIB).a $(LIB).so
```

La bandera `-fPIC` (*Position Independent Code*) es requerida para generar código que pueda cargarse en cualquier dirección de memoria, condición necesaria para las bibliotecas dinámicas `.so`. Las variables automáticas `$<`, `$@` y `$^` hacen las reglas genéricas y reutilizables.

src/Makefile:

```
CC     = gcc
CFLAGS = -Wall -I../include

.PHONY: all clean

all: ../bin/saludo_e ../bin/saludo_d

../bin/saludo_e: main.c
```

```
mkdir -p ../bin
$(CC) $(CFLAGS) $< ../lib/libhello.a -o $@

../bin/saludo_d: main.c
mkdir -p ../bin
$(CC) $(CFLAGS) $< -L../lib -lhello -o $@

clean:
rm -f ../bin/saludo_e ../bin/saludo_d
```

`saludo_e` se enlaza **estáticamente**: el código de la biblioteca queda embebido en el ejecutable (se pasa el `.a` directamente). `saludo_d` se enlaza **dinámicamente**: en tiempo de ejecución el SO carga `libhello.so` desde el path definido por `LD_LIBRARY_PATH`.

2.5. Pasos para construir

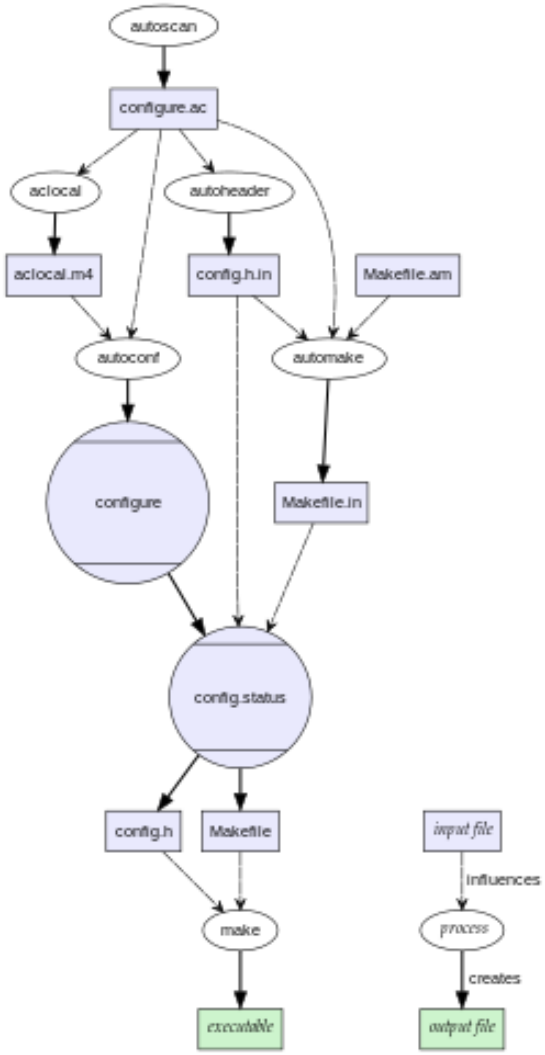
1. Crear la estructura y copiar los archivos fuente.
2. Desde el directorio raíz: `make`.
3. Verificar: `./bin/saludo_e`
4. Para el enlace dinámico, exportar la ruta antes de ejecutar:
   `export LD_LIBRARY_PATH=$(pwd)/lib && ./bin/saludo_d`
5. Limpiar: `make clean`.

3. Parte II: GNU Autotools

3.1. Conceptos

GNU Autotools (o GNU Build System) es un conjunto de herramientas que *generan automáticamente* los Makefiles y scripts de configuración necesarios para que un paquete sea portable entre distintas plataformas Unix. Sus tres componentes principales son:

- **Autoconf**: genera el script `configure` a partir de `configure.ac`. Este script verifica el compilador, las bibliotecas y otros aspectos del sistema antes de generar los Makefiles.
- **Automake**: genera `Makefile.in` a partir de `Makefile.am`. El script `configure` convierte `Makefile.in` en el `Makefile` final.
- **Libtool**: abstrae la creación de bibliotecas estáticas y dinámicas de forma portable entre plataformas.



Por Jdthood – CC BY-SA 3.0, `https://commons.wikimedia.org/w/index.php?curid=15581407`

3.2. Autotools: modo usuario

Antes de aplicar Autotools como desarrollador, conviene conocerlo en modo usuario instalando un paquete existente. Se utilizará `libpng` (`http://www.libpng.org/pub/png/libpng.html`), una biblioteca ampliamente usada para soporte del formato PNG.

1. Crear un directorio de trabajo y descargar el paquete:

```
mkdir ~/libpng && cd ~/libpng
wget https://sourceforge.net/projects/libpng/files/libpng16/1.6.39/libpng-1.6.39.tar.gz
```

2. Descomprimir y explorar el contenido:

```
tar -xvf libpng-1.6.39.tar.gz
cd libpng-1.6.39 && ls
```

   Se puede observar el script `configure` ya generado por Autoconf, listo para que el usuario lo ejecute.

3. Crear los directorios de construcción e instalación:

```
mkdir -p build/usr && cd build
```

4. Ejecutar el script de configuración con el prefijo de instalación local (se debe usar ruta absoluta):

```
../configure --prefix=$HOME/libpng/libpng-1.6.39/build/usr
```

   El script verifica el compilador, las dependencias del sistema y genera los Makefiles finales. Ejecutar `ls` para confirmar su creación.

5. Construir e instalar:

```
make && make install
```

6. Explorar el resultado en `build/usr/`: se crean los subdirectorios estándar `bin/`, `lib/`, `include/` y `share/`, equivalentes a la estructura de `/usr/` del sistema.

Este flujo es el que recibe cualquier usuario de un paquete Autotools. Como desarrollador, la tarea es generar ese mismo `configure` a partir de `configure.ac`, que es lo que se describe en la sección siguiente.

3.3. Aplicación: *libhello* con Autotools

La estructura con los archivos propios de Autotools es:

```
libhello/
  configure.ac          <- entrada de Autoconf
  Makefile.am           <- entrada de Automake (raiz)
  include/
    say.h
  lib/
    Makefile.am         <- entrada de Automake para la biblioteca
    say.c
  src/
```

```
Makefile.am         <- entrada de Automake para la aplicacion
main.c
```

configure.ac:

```
AC_INIT([libhello], [1.0], [correo@dominio.com])
AC_CONFIG_AUX_DIR([build-aux])
AC_CONFIG_MACRO_DIR([m4])
AM_INIT_AUTOMAKE([foreign -Wall -Werror])
AC_PROG_CC
AM_PROG_AR
LT_INIT
AC_CONFIG_HEADERS([config.h])
AC_CONFIG_FILES([Makefile lib/Makefile src/Makefile])
AC_OUTPUT
```

`LT_INIT` inicializa Libtool. `AC_CONFIG_MACRO_DIR([m4])` declara el directorio donde Libtool almacena sus macros. `AC_CONFIG_HEADERS([config.h])` hace que Autoconf genere un header con macros del sistema (como `PACKAGE_STRING`) que puede incluirse en el código fuente.

Makefile.am (raíz):

```
SUBDIRS = lib src
ACLOCAL_AMFLAGS = -I m4
```

lib/Makefile.am:

```
lib_LTLIBRARIES = libhello.la
include_HEADERS = $(top_srcdir)/include/say.h
AM_CPPFLAGS     = -I$(top_srcdir)/include
libhello_la_SOURCES = say.c
```

La variable `lib_LTLIBRARIES` indica a Libtool qué biblioteca construir y en qué directorio instalarla. El formato `.la` es el archivo de biblioteca de Libtool, que envuelve tanto la versión estática como la dinámica.

src/Makefile.am:

```
AM_CPPFLAGS      = -I$(top_srcdir)/include
bin_PROGRAMS     = saludo
saludo_SOURCES   = main.c
saludo_LDADD     = $(top_builddir)/lib/libhello.la
```

`saludo_LDADD` enlaza la aplicación con la biblioteca. Automake infiere automáticamente el tipo de enlace (estático o dinámico) según la configuración de Libtool.

3.4. Pasos para construir

1. Generar los scripts: `autoreconf --install`
2. `mkdir build && cd build`
3. Configurar con prefijo local:

   ```
   ../configure --prefix=$HOME/libhello/build/usr
   ```

4. `make`
5. `make install` — instala en `build/usr/{bin,lib,include}`

6. `./usr/bin/saludo`

7. Generar paquete distribuible: `make distcheck`
   Produce `libhello-1.0.tar.gz` listo para distribución y verifica que el paquete se construye desde cero correctamente.

4. Parte III: CMake

4.1. Conceptos

CMake es un sistema de construcción multiplataforma que genera los archivos de construcción nativos de cada plataforma (Makefiles en Linux, proyectos de Visual Studio en Windows, etc.) a partir de archivos `CMakeLists.txt`. Sus ventajas principales son:

- Independencia del compilador y la plataforma.
- Detección automática de dependencias.
- Integración con **CPack** para empaquetado y con **CTest** para pruebas.
- Construcción *out-of-source*: el árbol de construcción es completamente independiente del árbol de código fuente.

Cada directorio con código fuente debe contener un archivo `CMakeLists.txt`.

4.2. CMake: modo usuario

De forma similar a Autotools, el flujo de usuario es:

1. Descomprimir el paquete y crear un directorio `build/`.
2. `cmake ../ -DCMAKE_INSTALL_PREFIX:PATH=$PWD/usr`
3. `make && make install`

La diferencia respecto a Autotools es que no se distribuye un script `configure` precompilado: el usuario necesita CMake instalado.

4.3. Aplicación: *libhello* con CMake

La estructura con los archivos de CMake es:

```
libhello/
  CMakeLists.txt        <- archivo principal
  include/
    say.h
  lib/
    CMakeLists.txt      <- construye la biblioteca
    say.c
  src/
    CMakeLists.txt      <- construye la aplicacion
    main.c
```

CMakeLists.txt (raíz):

```
cmake_minimum_required(VERSION 3.10)
project(libhello VERSION 1.0.0 LANGUAGES C)

add_subdirectory(lib)
add_subdirectory(src)

# Empaquetado con CPack
```

```
set(CPACK_SOURCE_GENERATOR "TGZ")
set(CPACK_SOURCE_PACKAGE_FILE_NAME
    "${CMAKE_PROJECT_NAME}-${PROJECT_VERSION}")
set(CPACK_SOURCE_IGNORE_FILES "/build/;${CPACK_SOURCE_IGNORE_FILES}")
include(CPack)
```

lib/CMakeLists.txt:

```
include_directories(${CMAKE_SOURCE_DIR}/include)

set(LIB_NAME  hello)
set(INC_FILE  ${CMAKE_SOURCE_DIR}/include/say.h)

add_library(${LIB_NAME} SHARED say.c ${INC_FILE})

install(TARGETS ${LIB_NAME} DESTINATION lib)
install(FILES   ${INC_FILE} DESTINATION include)
```

`add_library(... SHARED ...)` declara la biblioteca como dinámica. CMake infiere automáticamente que se necesita `-fPIC` y gestiona el nombre del archivo resultante (`libhello.so`).

src/CMakeLists.txt:

```
include_directories(${CMAKE_SOURCE_DIR}/include)

add_executable(saludo main.c)
target_link_libraries(saludo hello)

install(TARGETS saludo DESTINATION bin)
```

`target_link_libraries` resuelve automáticamente tanto la dependencia de enlace como la de construcción: si la biblioteca no está construida aún, CMake la construye primero.

4.4. Pasos para construir

1. Instalación (si es necesario): `sudo apt-get install cmake`
2. `mkdir build && cd build`
3. Generar los Makefiles:

   ```
   cmake ../ -DCMAKE_INSTALL_PREFIX:PATH=$HOME/libhello/build/usr
   ```

4. `make`
5. `make install`
6. Exportar ruta y verificar:

   ```
   export LD_LIBRARY_PATH=$HOME/libhello/build/usr/lib
   ./usr/bin/saludo
   ```

7. Generar paquete de código fuente: `make package_source`
   Produce `libhello-1.0.0.tar.gz`.

5. Comparativa entre herramientas

Aspecto	**Make**	**Autotools**	**CMake**
Archivos de entrada	`Makefile`	`configure.ac` + `Makefile.am`	`CMakeLists.txt`
Generación de scripts	Manual (el dev escribe los Makefiles)	`autoreconf` genera `configure` + `Makefile.in`	`cmake` genera Makefiles (o proyectos de IDE)
Portabilidad	Baja (depende del shell y las herramientas del sistema)	Alta (Unix/Linux)	Muy alta (Linux, macOS, Windows)
Empaquetado	Manual (`tar`)	`make distcheck`	`make package_source` (CPack)
Cross-compilación	Cambiar `CC` manualmente	Flag `--host` en `configure`	Archivo *toolchain* de CMake
Curva de aprendizaje	Media	Alta	Media

6. Evaluación

6.1. Preguntas teóricas

1. ¿Cuál es la diferencia entre una regla explícita y una regla de patrón en Make? ¿Cuándo conviene usar cada una?
2. Explique la función de los archivos `configure.ac` y `Makefile.am` en el flujo de Autotools. ¿Cuál herramienta procesa cada uno?
3. ¿Por qué es importante el uso de macros de *toolchain* (`CC`, `AR`, `CFLAGS`) en Makefiles para proyectos de sistemas embebidos con cross-compilación?
4. ¿Qué ventaja ofrece la construcción *out-of-source* de CMake respecto al enfoque de Make?
5. Compare `make distcheck` (Autotools) con `make package_source` (CMake): ¿qué garantiza cada uno?

6.2. Ejercicio práctico

Desarrolle en lenguaje C una biblioteca llamada **liboperaciones** que exponga las siguientes cinco funciones matemáticas:

Función	**Prototipo sugerido**
Suma	`float op_suma(float a, float b)`
Resta	`float op_resta(float a, float b)`
Multiplicación	`float op_mult(float a, float b)`
División	`float op_div(float a, float b)`
Raíz cuadrada	`float op_sqrt(float a)`

Adicionalmente, desarrolle una aplicación **calculadora** que llame a cada función con valores de prueba e imprima los resultados.
Construya la biblioteca y la aplicación **con las tres herramientas** estudiadas en el taller, de acuerdo a los siguientes requisitos por herramienta:

- **Make**: Makefiles recursivos que generen la biblioteca estática (`liboperaciones.a`) y dinámica (`liboperaciones.so`), y la aplicación en sus dos variantes de enlace (`calculadora_e` y `calculadora_d`).
- **Autotools**: Archivos `configure.ac` y `Makefile.am` que construyan `liboperaciones.la` con Libtool. El paquete debe pasar `make distcheck` sin errores y producir el archivo `liboperaciones-1.0.tar.gz`.
- **CMake**: Archivos `CMakeLists.txt` que construyan la biblioteca como dinámica (`liboperaciones`) y generen el paquete de código fuente `liboperaciones-1.0.0.tar.gz` con CPack.

6.3. Estructura de entrega

```
apellido_nombre_taller3/
  teoria.pdf            <- respuestas a las preguntas teoricas
  make/
    liboperaciones/     <- estructura con Makefiles
  autotools/
    liboperaciones/     <- estructura con archivos Autotools
```

```
    liboperaciones-1.0.tar.gz
cmake/
  liboperaciones/     <- estructura con CMakeLists.txt
    liboperaciones-1.0.0.tar.gz
```

6.4. Entregable

- Único archivo `.tar` (con su nombre en el nombre del archivo) conteniendo la estructura descrita. Suba el archivo al TecDigital.

Material de referencia

- GNU Make: `https://www.gnu.org/software/make/manual/make.html`
- Autotools tutorial: `http://www.lrde.epita.fr/~adl/autotools.html`
- Automake manual: `http://www.gnu.org/software/automake/manual/automake.html`
- CMake documentation: `https://cmake.org/documentation/`
- CMake tutorial: `https://cmake.org/cmake/help/latest/guide/tutorial/index.html`